

1. IDENTIFICAÇÃO DO ANTEPROJETO DE PESQUISA

1.1. Título do anteprojeto

Benchmarking de técnicas de path tracing aplicadas em Vulkan e CUDA para renderização offline

1.2. Participantes:

Orientadores:

Mário Augusto Pazoti

Currículo Lattes: <http://lattes.cnpq.br/3530094254908684>

CPF: 286.965.538-02

Pesquisador:

José Vitor Vernize Martos

Currículo Lattes: <http://lattes.cnpq.br/2629273222538559>

CPF: 514.105.508-69

1.3. Área ou Linha de Pesquisa

Comparação entre estruturas de dados, técnicas e APIs para desenvolvimento de path tracers aplicados com foco em renderização offline.

1.4. Palavras-Chave

Path tracing, BVH, Kd-tree, Vulkan, CUDA.

1.5. Área de Conhecimento do Projeto (CNPQ)

1. Ciências Exatas e da Terra

1.03.00.00-7 Ciência da Computação

2. RESUMO

A simulação precisa da interação entre a luz e os objetos de uma cena constitui um dos principais desafios da computação gráfica contemporânea. Entre os métodos capazes de alcançar elevado grau de fidelidade, o path tracing destaca-se por modelar de forma abrangente os fenômenos de iluminação global. Contudo, a expressiva demanda computacional imposta por essa técnica requer o emprego de estratégias que otimizem o processo de interseção entre raios e superfícies. Diante disso, este trabalho propõe uma análise experimental que compara o desempenho de diferentes técnicas de aceleração, com destaque para as estruturas Bounding Volume Hierarchy (BVH) e K-dimensional Tree (Kd-tree), aplicadas a cenas com distintos níveis de complexidade geométrica. Além disso, são exploradas e contrastadas duas APIs amplamente utilizadas no desenvolvimento de renderizadores do gênero — Vulkan e CUDA — de forma a avaliar seu impacto sobre a eficiência computacional e a viabilidade da implementação. Essa comparação busca oferecer subsídios técnicos para a tomada de decisão na seleção de algoritmos e plataformas no desenvolvimento de soluções de renderização offline de alta fidelidade.

3. CONTEXTUALIZAÇÃO E PROBLEMÁTICA

A obtenção de imagens com alto grau de realismo visual representa um dos principais objetivos da computação gráfica, especialmente em aplicações que envolvem visualização científica, produção audiovisual e simulações virtuais. Dentre os métodos de renderização existentes, o path tracing, introduzido por Kajiya (1986), sobressai-se por sua capacidade de simular, com rigor físico, os fenômenos de iluminação global que ocorrem na interação entre luz e superfícies tridimensionais.

O path tracing, identificado como uma extensão do ray tracing, é considerado uma técnica extremamente poderosa por simular com precisão o comportamento físico da luz. Essa abordagem permite a renderização de materiais especulares e difusos, além de fenômenos visuais complexos como penumbras, cáusticas, transparências, névoa, entre outros, os quais seriam computacionalmente custosos em outros métodos de renderização. Contudo, conforme Akenine-Möller, Haines e Hoffman (2018), sua principal limitação se revela na elevada complexidade computacional. Para se alcançar imagens com qualidade visual cinematográfica, pode ser necessário traçar bilhões de caminhos.

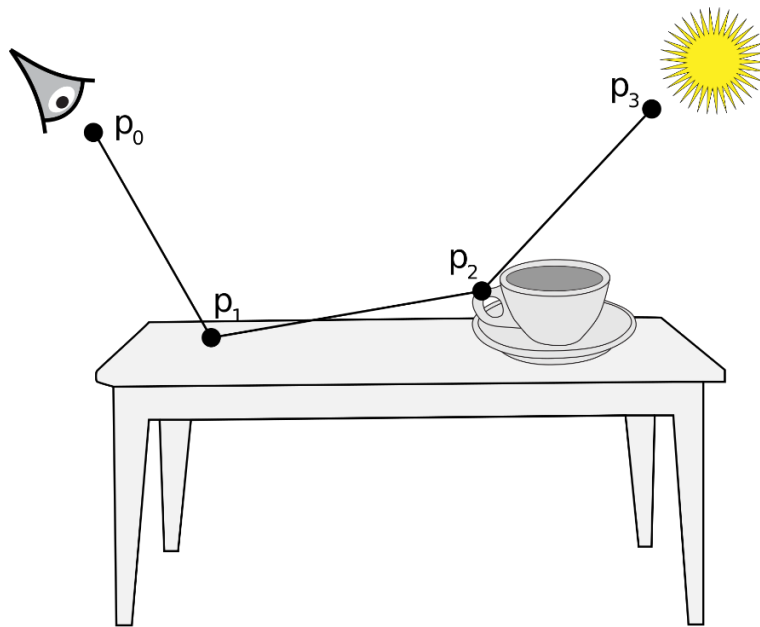


Figura 1: Representação do processo de path tracing. Adaptado de (PHARR, JAKOB e HUMPHREYS, 2023).

Dessa forma, o elevado custo computacional inerente à técnica, decorrente do grande número de interseções entre raios e elementos geométricos da cena, como ilustrado na Figura 1, impõe desafios significativos para sua aplicação eficiente em ambientes computacionais restritos.

3.1 Estruturas de Aceleração

Nesse sentido, desenvolveu-se estruturas de dados que possibilitam a aceleração do processo, dentre as proeminentes, a Bounding Volume Hierarchy (BVH) e a K-dimensional Tree (Kd-tree) têm sido amplamente empregadas, como reforçado por Vinkler, Havran e Bittner (2016), os quais compararam o desempenho de ambas as estruturas aplicadas em GPUs com o objetivo de reduzir o custo computacional do traçado de raios. Adiante, a elaboração de heurísticas para otimizar a qualidade dessas se tornou essencial, sendo a principal delas a *surface area heuristic* (SAH), a qual pode ser implementada em tanto em uma BVH quanto em uma Kd-tree.

- **Bounding Volume Hierarchy**

Bounding volume hierarchies (BVHs) são atualmente as estruturas de aceleração mais populares para traçar raios em GPU devido ao seu baixo consumo de memória e flexibilidade na adaptação a mudanças temporais na geometria da cena (KARRAS e AILA. 2013).

A estrutura consiste em um sistema hierárquico de volumes delimitadores que agrupam subconjuntos de primitivas da cena. A construção da BVH envolve a divisão recursiva do conjunto de primitivas em partições que minimizem a sobreposição entre volumes e maximizem a eficiência dos testes de interseção. Há também a vantagem de serem altamente paralelizáveis, um atributo essencial para a execução em GPUs.

Durante a fase de traçado, os raios percorrem a hierarquia testando primeiro os volumes pai antes de acessar os volumes filhos, o que reduz drasticamente a quantidade de interseções desnecessárias, conforme ilustrado na Figura 2.

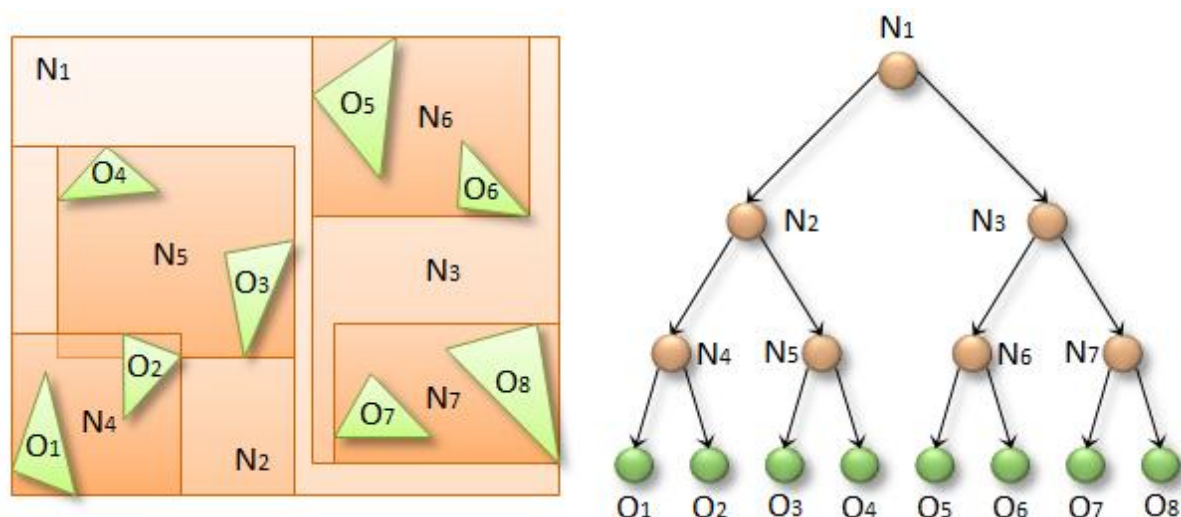


Figura 2: Representação de uma árvore BVH. Fonte: NVIDIA (2012).

- **K-dimensional Tree**

Por outro lado, a K-dimensional Tree (Kd-tree) representa uma abordagem baseada na divisão recursiva do espaço tridimensional por meio de planos ortogonais aos eixos coordenados, como ilustrado na Figura 3. Diferentemente da BVH, que organiza volumes, a Kd-tree particiona o espaço, classificando as primitivas de acordo com sua posição relativa aos planos de divisão.

Embora a Kd-tree seja uma opção eficiente para o traçado de raios, Choi *et al.* (2010) enfatizam uma desvantagem fundamental: O tempo de construção da Kd-tree proibitivamente caro. Esse custo não é razoável para renderizar cenas dinâmicas em sistemas *multicore* emergentes.

Logo, devido a construção da Kd-tree tender a ser mais custosa e menos adaptativa a mudanças dinâmicas na cena, torna-se uma alternativa relevante principalmente no contexto de renderizações offline.

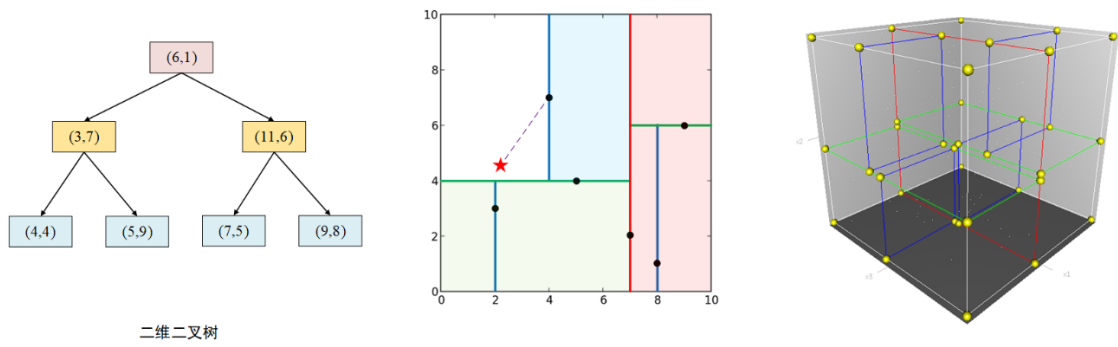


Figura 3: Representação de uma árvore Kd-tree. Fonte: CNBLOGS (2025).

- **Surface Area Heuristic**

A escolha do método de particionamento influencia significativamente a qualidade da árvore. A Surface Area Heuristic (SAH) considera a densidade da geometria no espaço 3D e elimina regiões vazias de forma eficiente (GARANZHA *et. al.*, 2011).

Essa heurística busca minimizar o custo esperado de interseções ao particionar o espaço tridimensional com base na geometria distribuída. Dessa forma, Arvo e Kirk (1989) apresentam seu princípio fundamental, o qual se baseia na suposição de que raios incidentes no espaço são distribuídos uniformemente e que a probabilidade de um raio intersectar uma determinada região é proporcional à área superficial dessa região.

A partir dessa premissa, a SAH fornece uma estimativa do custo esperado de interseção com objetos geométricos ao se realizar uma partição do espaço. Uma das formas de representar o custo de particionamento C de um nó é dado por:

$$C = C_T + C_I \left(\frac{SA_L}{SA} N_L + \frac{SA_R}{SA} N_R \right)$$

onde C_T representa o custo de travessia do nó, SA_L e SA_R são as áreas das superfícies dos volumes filhos L e R , respectivamente, SA é a área da superfície do nó atual, N_L e N_R correspondem aos números de triângulos dos volumes filhos L e R respectivamente.

3.2 APIs

A eficiência na execução de do path tracing depende da capacidade de explorar o paralelismo massivo das GPUs. Ao contrário das fases tradicionais do pipeline gráfico, os *Compute Shaders* e CUDA permitem a execução de tarefas de propósito geral na GPU, com maior versatilidade e controle sobre a organização dos dados e da execução paralela,

reduzindo o gargalo de transferência de dados com a CPU. Com o fito de utilizar essa abordagem, apresenta-se as APIs:

- **Vulkan**

A API Vulkan, mantida pelo Khronos Group, adota uma filosofia de baixo nível e multiplataforma, oferecendo suporte robusto tanto para renderização gráfica tradicional quanto para tarefas de computação paralela via *Compute Shaders*. Uma de suas características centrais é o uso do formato intermediário SPIR-V (*Standard Portable Intermediate Representation*), que atua como uma representação binária dos programas de *shader*. Vulkan (2025) apresenta que a vantagem de usar um formato de *bytecode* é que os compiladores escritos por fornecedores de GPU para transformar código de *shader* em código nativo são significativamente menos complexos. Além disso, o formato permite que linguagens de alto nível como GLSL ou HLSL sejam compiladas de forma padronizada, promovendo portabilidade e interoperabilidade entre diferentes fornecedores de hardware.

O suporte ao SPIR-V torna a API especialmente atrativa ao possibilitar que um mesmo código-fonte de *shader* seja reutilizado em dispositivos da NVIDIA, AMD, Intel e outros, além de conceder suporte a ray tracing acelerado por hardware. Nesse sentido, seu potencial também é notório em processamentos de propósito geral como evidenciado por Tolmachev (2023) ao ter desenvolvido VkFFT, uma biblioteca performática multiplataforma para cálculo de transformadas rápidas de Fourier em Vulkan.

Entretanto, Vulkan (2025) também menciona o aumento de responsabilidades da aplicação, verbosidade e trabalho advindos da proximidade ao hardware quando comparada a APIs gráficas de nível mais alto. Além disso, embora o SPIR-V proporcione portabilidade entre diferentes fabricantes de hardware, são impostas certas limitações em comparação a linguagens de computação dedicadas, como a ausência de suporte direto a ponteiros genéricos e pouco suporte a manipulação de estruturas dinâmicas de dados, o que pode dificultar a implementação de certos algoritmos. Tais restrições tornam necessário o uso de estratégias alternativas para representar estruturas como árvores, exigindo uma adaptação mais cuidadosa para sua execução.

- **CUDA**

A Compute Unified Device Architecture (CUDA), desenvolvida pela NVIDIA, é uma plataforma especializada em programação paralela, Ghorpade *et. al.* (2012) referenciam

a sua ampla adoção para aplicações científicas de alto desempenho e é demonstrado por Karras e Aila (2013) o potencial dessa API para construção de BVHs.

A CUDA oferece uma interface de programação de alto nível, com acesso direto a recursos internos da GPU. Uma das suas características atrativas é sua integração com a linguagem C++ e possuir uma consolidada série de bibliotecas, o que permite que os desenvolvedores aproveitem a flexibilidade da linguagem, ao mesmo tempo em que utilizam o poder de computação das unidades de processamento gráfico.

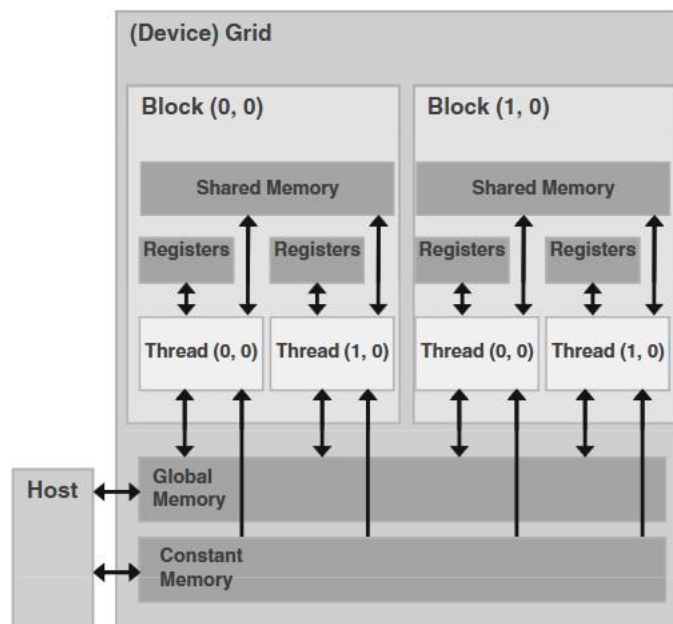


Figura 4: Representação do modelo de memória da CUDA. (KIRK e HWU. 2016)

Cada kernel, a unidade de execução de um programa CUDA, pode ser distribuído entre milhares de threads. Em sua estrutura hierárquica de blocos e threads, como visualizado na Figura 4, é possível notar a necessidade de possuir controle sobre a alocação de memória e suas regiões para assim extrair o máximo de proveito que a plataforma dispõe.

Essa abordagem permite otimizações específicas e refinadas, resultando em desempenho excepcional quando utilizada em hardware compatível. No entanto, sua aplicação está restrita a GPUs da NVIDIA, o que limita sua adoção em cenários que demandam portabilidade entre as demais arquiteturas.

4. JUSTIFICATIVA

A relevância deste trabalho reside na análise comparativa de diferentes abordagens voltadas à construção de um modelo computacional eficiente e eficaz para a renderização de

offline, as quais desempenham papel fundamental em distintos setores da indústria e do meio acadêmico, como arquitetura, engenharia, cinema e visualização científica. Dentre as diversas técnicas existentes, o path tracing desponta como uma das mais promissoras, especialmente por sua capacidade de simular com elevado grau de realismo os fenômenos de iluminação global.

A contribuição deste estudo está em oferecer uma base sólida para a tomada de decisão técnica por parte de desenvolvedores e pesquisadores que atuam na área de computação gráfica. Ao comparar estruturas de aceleração e as APIs, o trabalho possibilita a identificação de soluções mais adequadas às demandas específicas de cada cenário, promovendo ganhos em desempenho e qualidade.

Além disso, ao integrar múltiplos aspectos técnicos sob uma perspectiva experimental e sistemática, o estudo amplia o corpo de conhecimento disponível sobre o tema, contribuindo para o avanço da pesquisa em renderização computacional e para a formação de uma visão crítica e fundamentada sobre as tecnologias envolvidas.

5. OBJETIVOS

Analisar e comparar o desempenho de diferentes técnicas e estruturas de aceleração aplicadas ao path tracing em cenários de renderização offline, com foco na eficiência computacional e na viabilidade de implementação utilizando as APIs Vulkan e CUDA. A fim de atingir o objetivo geral pretende-se:

- Investigar os fundamentos teóricos e práticos do path tracing.
- Estudar e implementar estruturas de dados utilizadas para aceleração do processo de interseção, com foco nas abordagens com BVHs e Kd-trees.
- Comparar as APIs Vulkan e CUDA em suas capacidades computacionais e práticas, analisando sua adequação ao desenvolvimento de path tracers eficientes.
- Realizar experimentos com diferentes configurações de cena, variando a complexidade geométrica, para avaliar o impacto das estruturas e APIs sobre o desempenho da renderização.
- Discutir os resultados obtidos à luz de métricas quantitativas como tempo de construção das estruturas, tempo total de renderização, memória e escalabilidade, além de fatores qualitativos como a fluidez da implementação e depuração.
- Oferecer subsídios técnicos e práticos que auxiliem na escolha de tecnologias mais adequadas para aplicações específicas de renderização offline.

6. RESULTADOS ESPERADOS

O trabalho visa responder a hipóteses técnicas frequentemente levantadas na prática de desenvolvimento de renderizadores, o que pretende validar, com base empírica, quais tecnologias e estratégias oferecem melhores resultados sob diferentes condições. Espera-se, assim, que os experimentos realizados forneçam evidências sobre o impacto das diferentes estruturas de aceleração e APIs gráficas na eficiência do path tracing.

Entre os resultados esperados, destacam-se:

- A identificação de combinações mais eficazes de técnicas através de dados quantitativos de desempenho computacional, considerando métricas como tempo de construção das estruturas, tempo médio de renderização, uso de memória e escalabilidade.
- A caracterização dos limites de escalabilidade de cada abordagem, por meio da análise do comportamento do sistema através de ferramentas como o RenderDoc (2025) e NVIDIA Nsight (2025) diante do aumento da complexidade das cenas em número de objetos, profundidade dos raios e resolução da imagem.
- Avaliação comparativa qualitativa entre Vulkan e CUDA no que se refere à facilidade de implementação, depuração, controle sobre hardware gráfico e integração com estruturas de aceleração.
- A produção de materiais técnicos e analíticos como gráficos e tabelas comparativos que auxiliem desenvolvedores e pesquisadores na seleção de APIs e estruturas de aceleração mais adequadas às suas necessidades específicas.

Tais resultados visam não apenas contribuir para a fundamentação teórica da área, mas também oferecer aplicabilidade prática, fornecendo parâmetros técnicos para decisões de projeto e desenvolvimento em renderização offline. A expectativa é que os dados gerados sirvam de referência para futuros estudos, adaptações industriais e aplicações em pipelines gráficos voltados a implementação de path tracers.

7. METODOLOGIA

A metodologia adotada neste trabalho está estruturada em etapas sequenciais com enfoque comparativo. Inicialmente, será realizada uma revisão bibliográfica aprofundada

sobre os principais conceitos relacionados à renderização baseada em path tracing, e características técnicas das estruturas de aceleração (BVH e Kd-tree) e as APIs (Vulkan e CUDA), com o objetivo de embasar teoricamente as decisões de implementação.

Após a revisão, em primeiro momento, serão desenvolvidas implementações fundamentais de um path tracer, utilizando Vulkan para iniciar o desenvolvimento com shaders SPIR-V. Para cada versão, serão integradas as estruturas de aceleração mencionadas, de modo a permitir a troca modular entre BVH e Kd-tree durante os testes. As estruturas serão construídas utilizando a heurística SAH e embasamento em algoritmos consagrados na literatura conforme: Wald (2007) para SAH-BVH e Hapala e Havran (2011) para SAH-Kd-tree, o que proporciona os métodos necessários para permitir o particionamento e intersecção dos raios. Adiante, serão avaliados corretude e desempenho das estruturas a partir de testes unitários e benchmarks preliminares de tempo de construção e eficiência em cenas com geometrias conhecidas, como a Cornell Box, concebida por Goral et al. (1984).

Em segundo momento, far-se-á o desenvolvimento em CUDA e serão definidos cenários de teste com diferentes níveis de complexidade geométrica e variações em fatores como número de objetos, profundidade de recursão dos raios e resolução de imagem em ambas as implementações. Esses cenários serão utilizados para realizar experimentos controlados, nos quais serão coletadas as métricas quantitativas:

- Tempo de construção das estruturas de aceleração.
- Tempo médio de renderização por frame.
- Uso de memória.
- Escalabilidade de tempo e memória conforme o aumento da complexidade da cena.

A coleta de dados será realizada com apoio de ferramentas de *debugging*, como o RenderDoc, software open-source que permite depurar aplicações em diferentes fornecedores de GPUs, e *profiling* com NVIDIA Nsight para uma análise mais aprofundada em placas gráficas NVIDIA, além da análise de desempenho por meio de temporizadores de CPU e GPU, nativos das APIs (vkGetQueryPoolResults e cudaEventElapsedTime), embutidos via código.

Os resultados obtidos serão comparados quantitativamente por meio de gráficos e tabelas visando identificar padrões de desempenho e *trade-offs* entre as diferentes combinações de estruturas e APIs, seguido de uma discussão qualitativa sobre os aspectos pertinentes a implementação, como a complexidade do código fonte resultante decorrente do número de linhas, além do suporte das APIs nos quesitos de gestão de memória, necessidade

de *workarounds* para o acesso das estruturas de aceleração e fluidez da depuração em prol da conquista dos objetivos descritos.

Destarte, será feita a compilação das observações e resultados na elaboração de diretrizes que auxiliem escolher uma estrutura ou API em detrimento de outra conforme o contexto do projeto. Dessa forma, permite-se a síntese de subsídios técnicos de forma clara, objetiva e coerente com os dados obtidos.

8. EQUIPAMENTO E MATERIAL

Para o desenvolvimento do projeto será utilizado um notebook pessoal, fazendo o uso das linguagens: C++ e GLSL, APIs: Vulkan e CUDA, e bibliotecas auxiliares: GLFW e GLM. A partir disso, os testes serão realizados em diversos outros hardwares para promover uma gama de benchmarks e enriquecer as comparações.

9. CRONOGRAMA

	set/25	out/25	nov/25	dez/25	jan/26	fev/26	mar/26	abr/26	mai/26	jun/26	jul/26	ago/26
1	x	x	x									
2	x	x	x									
3		x	x									
4		x	x	x								
5			x	x								
6				x	x							
7						x	x					
8					x	x	x	x				
9								x	x	x		
10											x	x

Legenda

1. Levantamento bibliográfico: path tracing, estruturas de aceleração, CUDA e Vulkan
2. Estudo aprofundado das técnicas BVH e Kd-tree; análise de trabalhos relacionados

3. Definição da proposta experimental, escopo da comparação e critérios de avaliação
4. Início da implementação do path tracer base
5. Implementação da estrutura BVH em Vulkan.
6. Implementação da estrutura Kd-tree na mesma API para efeito comparativo
7. Implementação do path tracer em uma segunda API.
8. Execução de experimentos com cenas de diferentes complexidades
9. Análise dos resultados e interpretação comparativa
10. Redação do relatório e apresentação dos resultados

Referências

KAJIYA, James T. The rendering equation. In: *Proceedings of the 13th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '86)*. New York: ACM, 1986. p. 143–150. DOI: <https://doi.org/10.1145/15922.15902>.

AKENINE-MÖLLER, Tomas; HAINES, Eric; HOFFMAN, Naty. *Real-time rendering*. 4. ed. Boca Raton: CRC Press, 2018. ISBN 978-1138627000.

PHARR, Matt; JAKOB, Wenzel; HUMPHREYS, Greg. *Physically based rendering: from theory to implementation*. 4. ed. Cambridge: The MIT Press, 2023. ISBN 978-0262048026.

VINKLER, M.; HAVRAN, V.; BITTNER, J. Performance comparison of bounding volume hierarchies and kd-trees for GPU ray tracing. *Computer Graphics Forum*, v. 35, n. 8, p. 68–79, 2016. DOI: <https://doi.org/10.1111/cgf.13020>.

KARRAS, Tero; AILA, Timo. Fast parallel construction of high-quality bounding volume hierarchies. In: *Proceedings of the 5th High-Performance Graphics Conference (HPG '13)*. New York: ACM, 2013. p. 89–99. DOI: <https://doi.org/10.1145/2492045.2492055>.

NVIDIA. *Thinking Parallel, Part II: Tree Traversal on the GPU*. 2012. Disponível em: <https://developer.nvidia.com/blog/thinking-parallel-part-ii-tree-traversal-gpu/>. Acesso em: 25 abr. 2025.

CHOI, Byn et al. Parallel SAH k-D tree construction. In: *Proceedings of the Conference on High Performance Graphics (HPG '10)*. Goslar: Eurographics Association, 2010. p. 77–86. DOI: <https://doi.org/10.2312/EGGH/HPG10/077-086>.

CNBLOGS. Disponível em: <https://img2022.cnblogs.com/blog/1602810/202207/1602810-20220705173948686-1623868865.png>. Acesso em: 25 abr. 2025.

GARANZHA, Kirill; PREMOŽE, Simon; BELY, Alexander; GALAKTIONOV, V. Grid-based SAH BVH construction on a GPU. *The Visual Computer*, [S.l.], v. 27, n. 6, p. 697–706, 2011. DOI: <https://doi.org/10.1007/s00371-011-0593-8>

ARVO, James; KIRK, David. A survey of ray tracing acceleration techniques. Em: GLASSNER, Andrew S. (Ed.). *An Introduction to Ray Tracing*. San Diego: Academic Press, 1989. cap. 6, p. 201–262. ISBN 978-0122861604.

TOLMACHEV, Dmitrii. VkFFT and beyond: a platform for runtime GPU code generation. In: *Proceedings of the 2023 International Workshop on OpenCL (IWOCCL '23)*. New York: ACM, 2023. Article 9. DOI: <https://doi.org/10.1145/3585341.3585357>.

VULKAN. *Shader modules*. Disponível em: https://docs.vulkan.org/tutorial/latest/03_Drawing_a_triangle/02_Graphics_pipeline_basics/01_Shader_modules.html. Acesso em: 09 mai. 2025.

VULKAN. *What is Vulkan?* Disponível em: https://docs.vulkan.org/guide/latest/what_is_vulkan.html. Acesso em: 09 mai. 2025.

GHORPADE-AHER, Jayshree; PARANDE, Jitendra; KULKARNI, Madhura; BAWASKAR, Amit. GPGPU processing in CUDA architecture. 2012. Disponível em: https://www.researchgate.net/publication/220484926_GPGPU_processing_in_CUDA_architecture. Acesso em: 27 abr. 2025.

KIRK, David B.; HWU, Wen-mei W. *Programming massively parallel processors: a hands-on approach*. 4. ed. Cambridge: Elsevier, 2022. ISBN 978-0323912310.

I. Wald, On fast Construction of SAH-based Bounding Volume Hierarchies, *2007 IEEE Symposium on Interactive Ray Tracing*, Ulm, Germany, 2007, pp. 33-40, DOI: 10.1109/RT.2007.4342588.

Hapala, M. and Havran, V. Review: Kd-tree Traversal Algorithms for Ray Tracing. *Computer Graphics Forum*, 30: 199-213. 2011. DOI: <https://doi.org/10.1111/j.1467-8659.2010.01844.x>

Cindy M. Goral, Kenneth E. Torrance, Donald P. Greenberg, and Bennett Battaile. Modeling the interaction of light between diffuse surfaces. *SIGGRAPH Comput. Graph.* 18, 3 (July 1984), 213–222. 1984. DOI: <https://doi.org/10.1145/964965.808601>

BALDWIN, G. *RenderDoc – Graphics debugger*. Disponível em: <https://renderdoc.org/> . Acesso em: 24 ago. 2025.

NVIDIA. *NVIDIA Nsight Graphics*. Disponível em: <https://developer.nvidia.com/nsight-graphics>. Acesso em: 24 ago. 2025.